EGCO 425 – Chapter 6

Classification Methods

- Artificial Neural Network (ANN)
- Support Vector Machine (SVM)

References

[1] Tan et al., Introduction to Data Mining (Chapter 5)

[2] Han & Kamber, Data Mining: Concepts and Techniques (Chapter 9)

[3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 4)

[4] RapidMiner Doc, https://docs.rapidminer.com/latest/studio/operators/

R. Marukatat

Chapter Outline

- Neural network
  - Perceptron & multi-layer perceptron (MLP)
  - Activation function
  - Back propagation training, gradient descent
  - Parameters : learning rate, momentum
- Support vector machine (SVM)
  - Hyperplane
  - Kernel function

Artificial Neural Network (ANN)

- Mimic biological nervous system

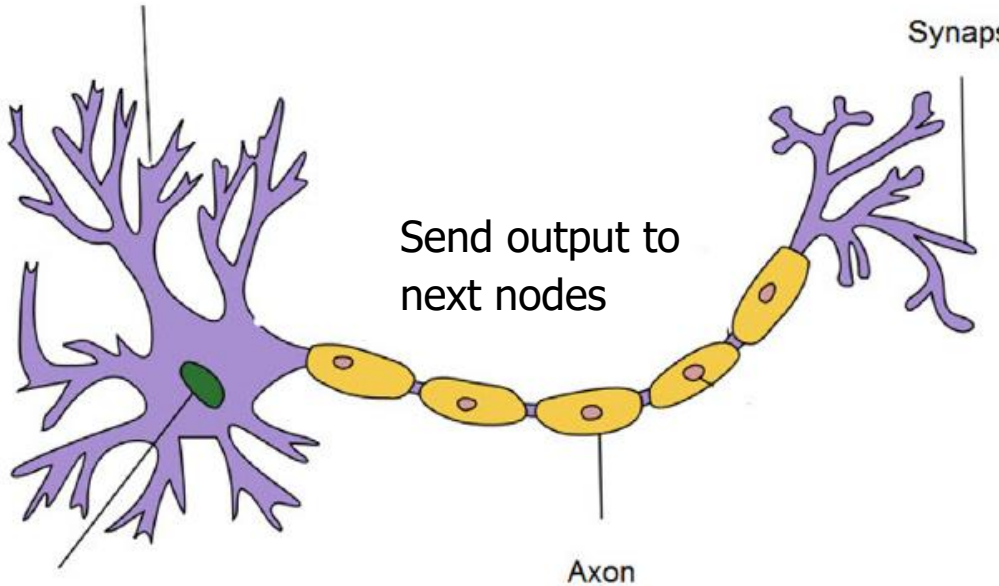
= input links

Dendrite

Connect to dendrites of other nodes

Synapse

Get inputs from many previous nodes



Send output to next nodes

Axon

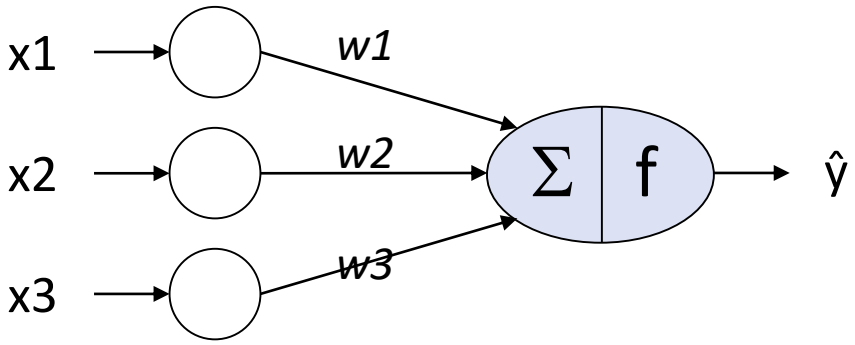
Nucleus

= output link

= node in ANN

Single Perceptron

- Suppose we have 3 attributes $(x1, x2, x3)$ and class y
- Architecture = 3 input nodes + 1 output node (= perceptron)



- Links between input nodes and output node have different weights (weights are usually randomed in the beginning)
- Input nodes read each training record $(x1, x2, x3)$ and pass the values to output nodes ➔ no computation in input layer
- Output node or perceptron has 2 functions
  - Summation function calculates $sum = \sum w_i x_i - bias$
  - Activation function predicts $\hat{y} = f(sum)$

Example : calculating perceptron's output

- Suppose that
  - Weight vector : $W = (w_1, w_2, w_3) = (0.3, 0.3, 0.3)$
  - $\text{Bias} = 0.4$
  - $\text{Sum} = 0.3x_1 + 0.3x_2 + 0.3x_3 - 0.4$
  - Activation function = sign function where $\hat{y} = \begin{cases} 1 & ; sum > 0 \\ -1 & ; sum \leq 0 \end{cases}$
- Consider training records (x_1, x_2, x_3, y)
  - $(1, 0, 0, -1)$ → $\hat{y} = -1$ correct prediction
  - $(1, 0, 1, -1)$ → $\hat{y} = 1$ wrong prediction
  - $(1, 1, 0, 1)$ → $\hat{y} = 1$ correct prediction

Training a perceptron

Initialize weights with random values

For each training cycle (epoch) {

For each training record with known class label y {

1. Feed training record to the perceptron to predict $\hat{y}$
2. Calculate $error = y - \hat{y}$
3. Adjust each weight $w_{i,new} = w_{i,current} + \eta\,(error)\,x_i$

}

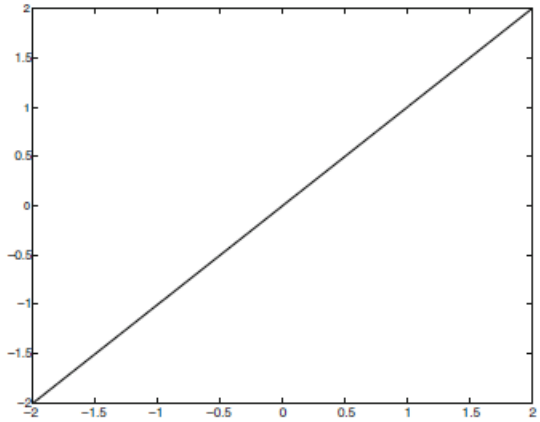
Complete 1 cycle (i.e. all training records are processed) ➔ calculate training error

} until stopping criteria e.g. weights are stable, training error < threshold, #cycles completed

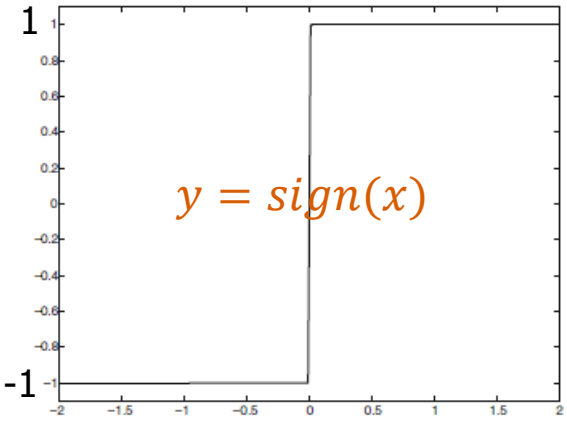
Classification model = perceptron model with optimal weights

Activation Function

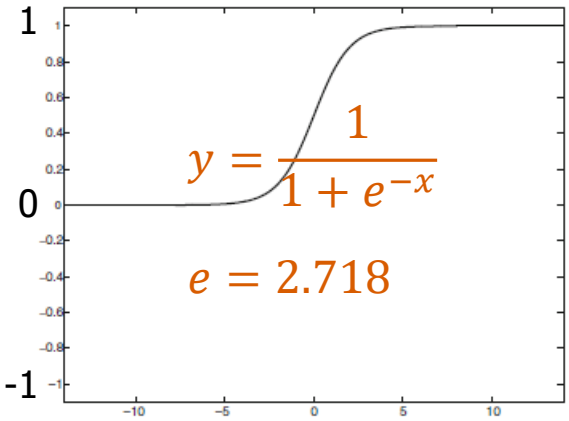
- Add nonlinearity to the model. Popular functions are
  - Identity (linear), sign
  - Sigmoid, tanh (hyperbolic tangent or scaled Sigmoid)
  - ReLU (rectified linear), hard tanh
- Main characteristic of these functions (except identity function) is that they map input of any range to bounded output of [0, 1] or [-1, 1] range



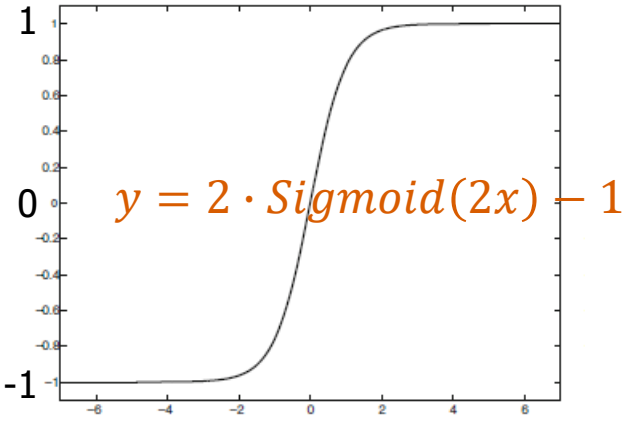
(a) Identity



(b) Sign



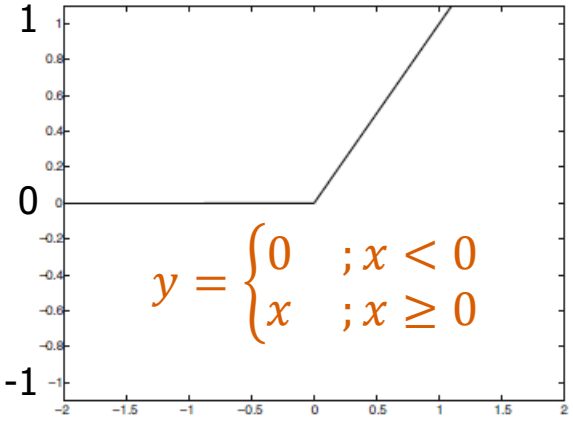
(c) Sigmoid



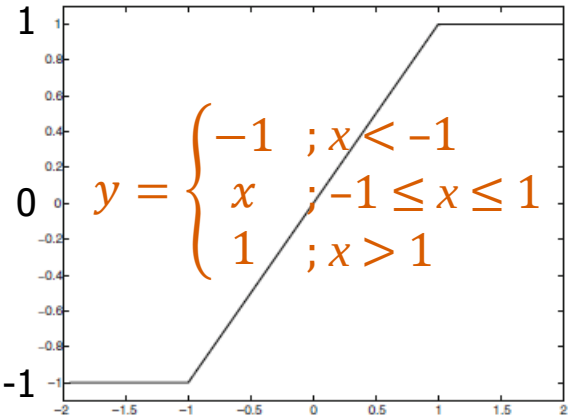
(d) Tanh

Tanh's curve is steeper than Sigmoid

So training converges faster than Sigmoid



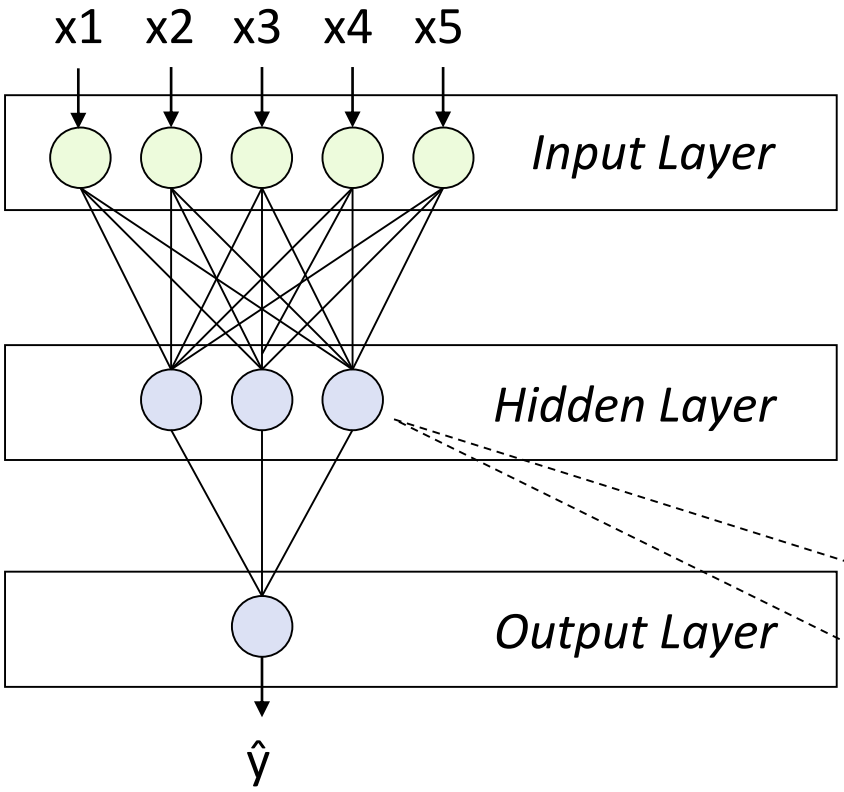
(e) ReLU



(f) Hard Tanh

ReLU and hard tanh are popular in deep learning network as they are easier and faster to calculate

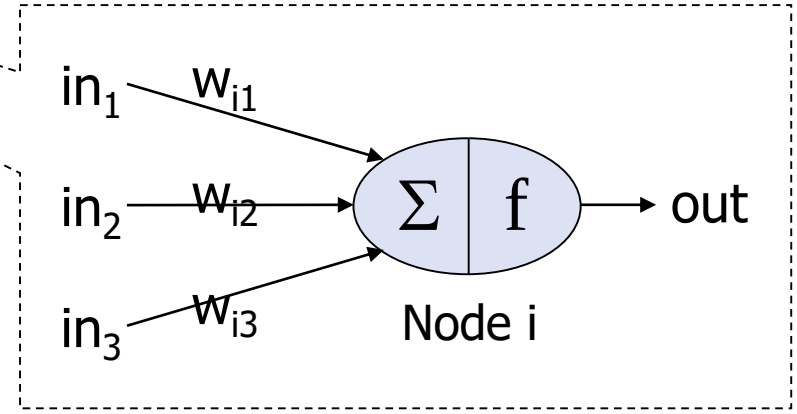
Multi-Layer Perceptron (MLP)



2-layer feed-forward ANN
(no computation in input layer)

There are many types of ANN, but we focus on

- Feed-forward ➔ nodes in one layer connect to nodes in the next layer only
- N-layer ➔ computation is done in N (excluding input) layers

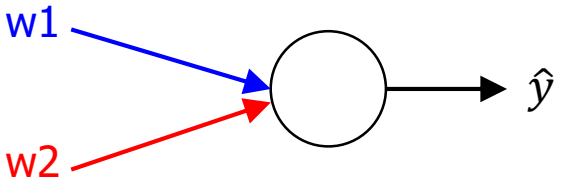


Back Propagation Training

- Training MLP is similar to training a single perceptron, but weight adjustment is more complicated
  - We see only final prediction $\hat{y}$ from the output layer
  - But we don't know output from hidden layer ➔ estimate how much each node in each layer contributes to the error & adjust weights accordingly
- To process each record (i.e. each round of the inner loop in slide 6)
  - Forward phase ➔ feed input values through MLP until we get $\hat{y}$
  - Backward phase ➔ propagate error $y - \hat{y}$ backward, estimate the contribution of each node & adjust weights
  - So weights are adjusted from back to front

Gradient Descent

- Error at output node can be thought of as a function of weights (i.e. the contribution of all weights). So it is a surface in multi-dimensional space



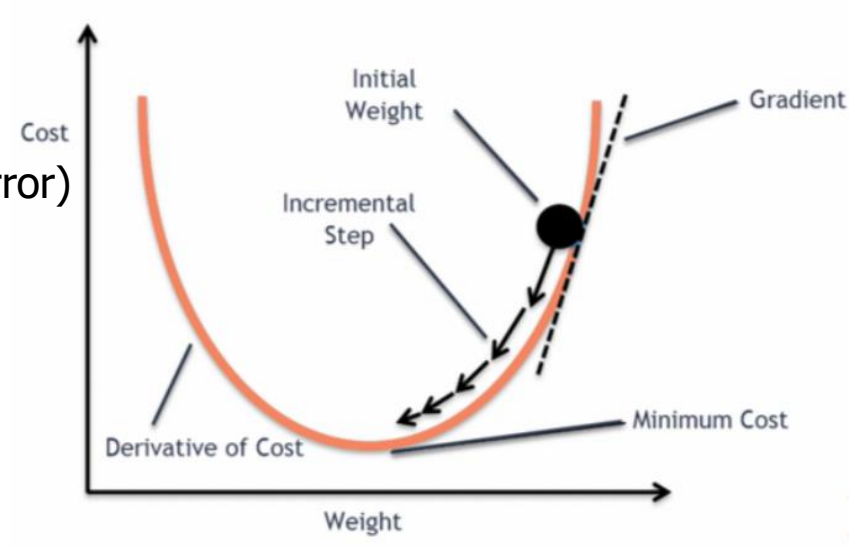
Given weight vector $W = (w_1, w_2)$

$error(W) = f(w_1, w_2)$

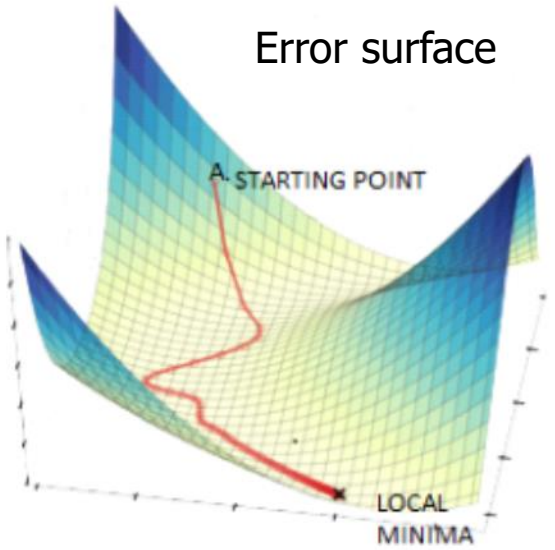
- Weight adjustment is based on gradient descent concept
  - Find slope or gradient of current point w.r.t. each weight dimension

$$\frac{\partial\, error(W)}{\partial\, w_1} ; \frac{\partial\, error(W)}{\partial\, w_2}$$

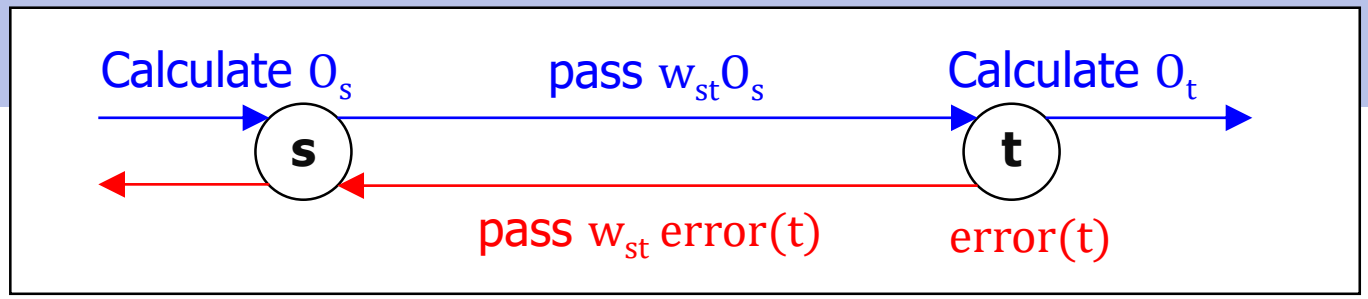
  - Walk along that direction toward the point where error is minimum



Weight adjustment of each weight
(1-dimensional view)



Weight adjustment of all weights
(multi-dimensional view)



Error at output node $= error(W) = \frac{1}{2}(y - f(W))^2$

Power 2 is for squared error. Factor ½ is added for easy derivative

Contribution of node **i** to the total error via link $\boldsymbol{w_{ij}}$

$$error(i) = \frac{\partial\, error(W)}{\partial\, w_{ij}} = (A_i - O_i)\frac{d\, f(W)}{d\, w_{ij}} = (A_i - O_i)\big(O_i(1 - O_i)\big)$$

A_i = actual value (unknown for hidden node) ; O_i = output from $f(W)$

If activation function = Sigmoid,

$$f(x) = \frac{1}{1 + e^{-x}} \quad \rightarrow \quad \frac{d\, f(x)}{dx} = f(x)\big(1 - f(x)\big) = O_i(1 - O_i)$$

Depend on activation function

From previous slide $error(i) = \big(O_i(1-O_i)\big)\,(A_i - O_i)$

If **i** = output node $error(i) = \big(O_i(1-O_i)\big)\,(y - O_i)$

If **i** = hidden node $error(i) = \big(O_i(1-O_i)\big)\,\sum_j w_{ij}\ error(j)$

First term = gradient of activation fn

Second term = error

Recall perceptron model $w_{i,new} = w_{i,current} + \eta\ (error)\ x_i$

Apply this to MLP $w_{ij,new} = w_{ij,current} + \eta\ \big(error(j)\big)(O_i)$

The full formula is $w_{ij,new} = w_{ij,current} + \Delta w_{ij,current}$

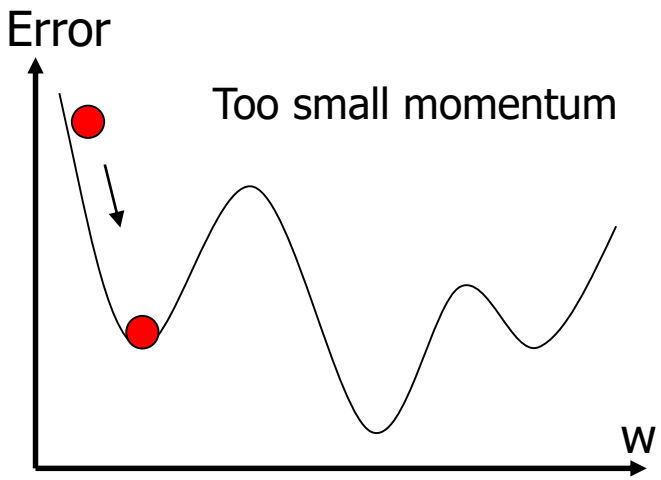
$$\Delta w_{ij,current} = \underbrace{\boldsymbol{\eta}\ \big(error(j)\big)(O_i)}_{\text{Due to current prediction}} + \underbrace{\boldsymbol{\alpha}\ \Delta w_{ij,previous}}_{\text{Momentum from previous adjustment}}$$

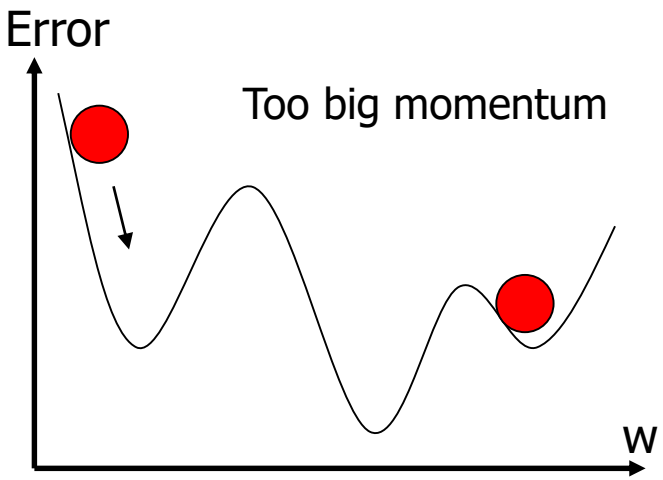
Learning rate (η)

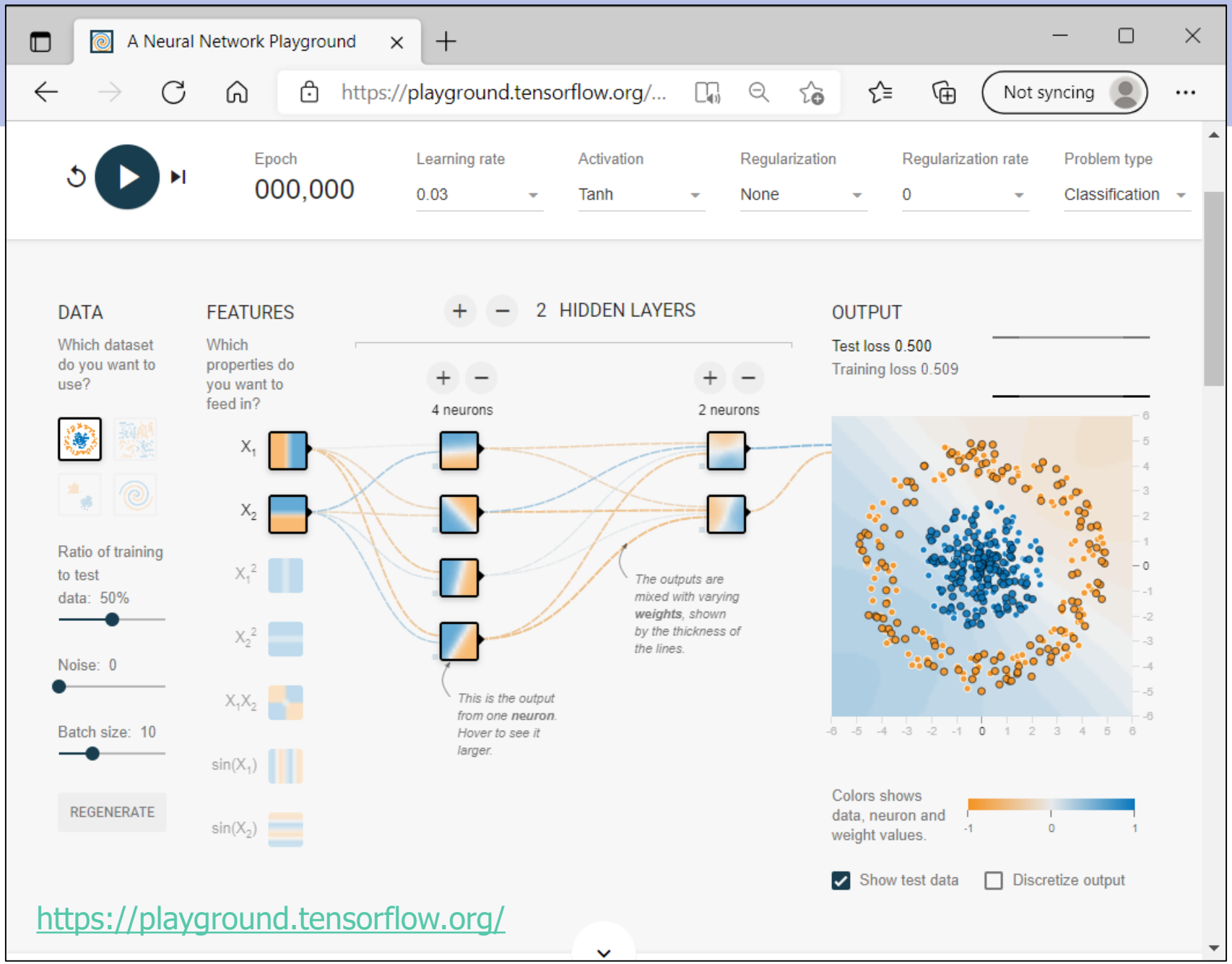
- $0 < \eta < 1$
  - Control the amount of each weight adjustment
  - Too small η new weight is close to old weight, slow convergence
  - Too big η new weight is sensitive to the current prediction, oscillation or overshooting optimal point
- Adaptive learning rate
  - Assign large η in the first few training cycles, so weights are quickly adjusted to near optimal point
  - Decrease η in subsequent cycles, so weights are fine-tuned to reach optimal point

Momentum (α)

- $0 < \alpha < 1$
  - Smoothen weight adjustment
  - By keeping the momentum from previous adjustment → so that the current adjustment doesn't sharply swing to the opposite direction
  - Too small α not enough momentum to reach global optimum (stuck at local optimum)
  - Too big α overshooting global optimum





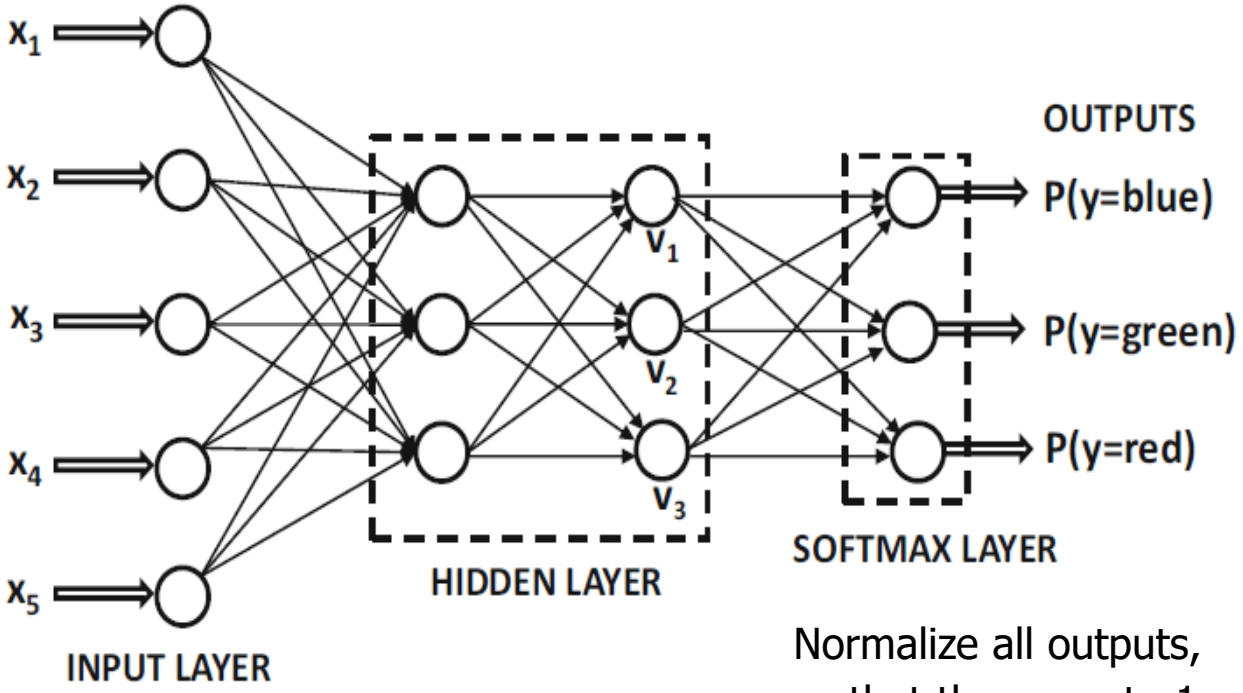


https://playground.tensorflow.org/

ANN Configuration

- Input layer
  - Most ANN implementations expect only numeric values in [0, 1] or [-1, 1] range. So normalization may be required
    - In RapidMiner, normalization can be set as a parameter in neural network (no need for separate preprocessing)
  - Numeric attribute ➔ 1 input node for each attribute
  - Nominal attribute ➔ k input nodes for each attribute
    - Require type conversion
    - A proper approach is to apply binarization first (see Chapter 3)
      From nominal to binary ➔ k binary attributes for k categories
      From binary to numeric ➔ k numeric attributes with values 0, 1

- Output layer
  - Usually 1 output node for each class
  - Most ANN implementation binarizes and converts class attribute to numeric internally
    - Suppose that class = {red, green, blue}
    - If a training record has class label = red ➔ its actual output would be {1, 0, 0}
  - For predicted output ➔ softmax function is usually applied to convert each value to the probability of each class



Normalize all outputs, so that they sum to 1 & can be interpret as class probabilities

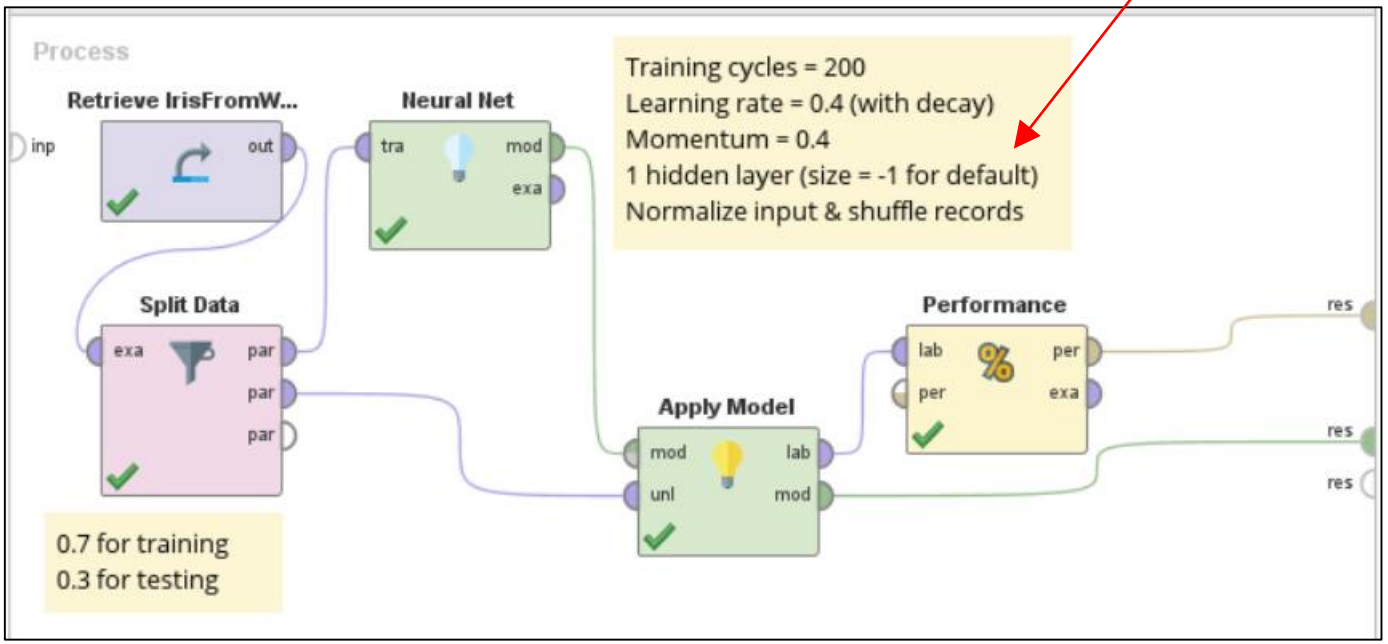
- Hidden layers
  - Number of layers
    - 1 layer enough for most classification tasks
    - 2 layers suitable for function fitting tasks
    - >2 layers not recommended because training is too difficult
    - Too many layers may give worse, not better, performance
  - Number of nodes in each layer
    - Should be between the sizes of input and output layers
    - Typical size ≈ (#input nodes + #output nodes) / 2

Parameters vs. Hyperparameters

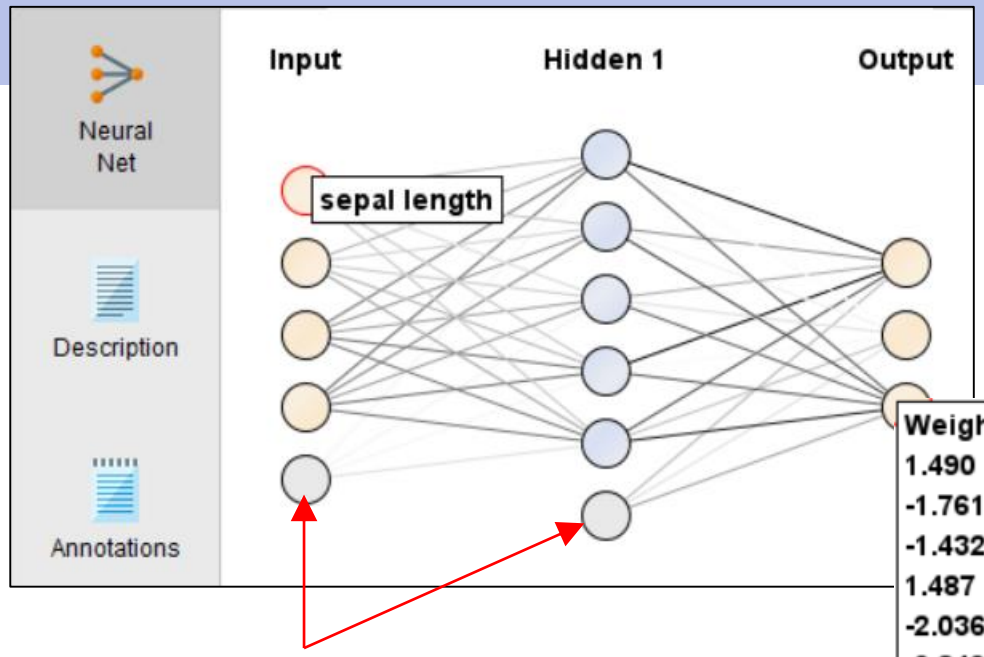
- Parameters
  - Internal to the model
  - They are adjusted during the training ➔ to obtain optimal parameters & hence optimal model that fits the data
  - E.g. weights in neural network
- Hyperparameters
  - External to the model
  - They are set before the training to control the learning process & are not part of resulting model (that fits the data)
  - E.g. #hidden layers, activation functions, learning rate, momentum

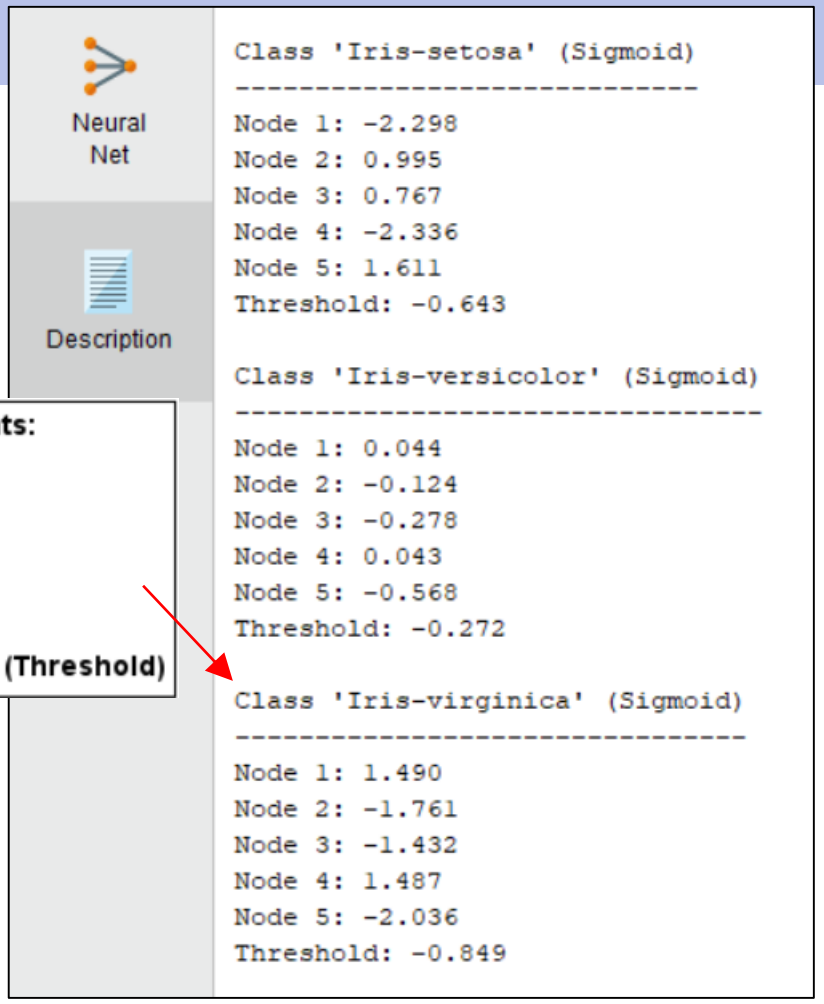
Example (1) : neural net with numeric attributes

$$default\ size = \left\lceil \frac{\#attributes + \#classes}{2} \right\rceil + 1$$



- RapidMiner uses scaled Sigmoid as activation function
- If records are sorted and the ones exhibiting certain patterns are clustered together, the model may be overfitting ➔ shuffle records to avoid this case





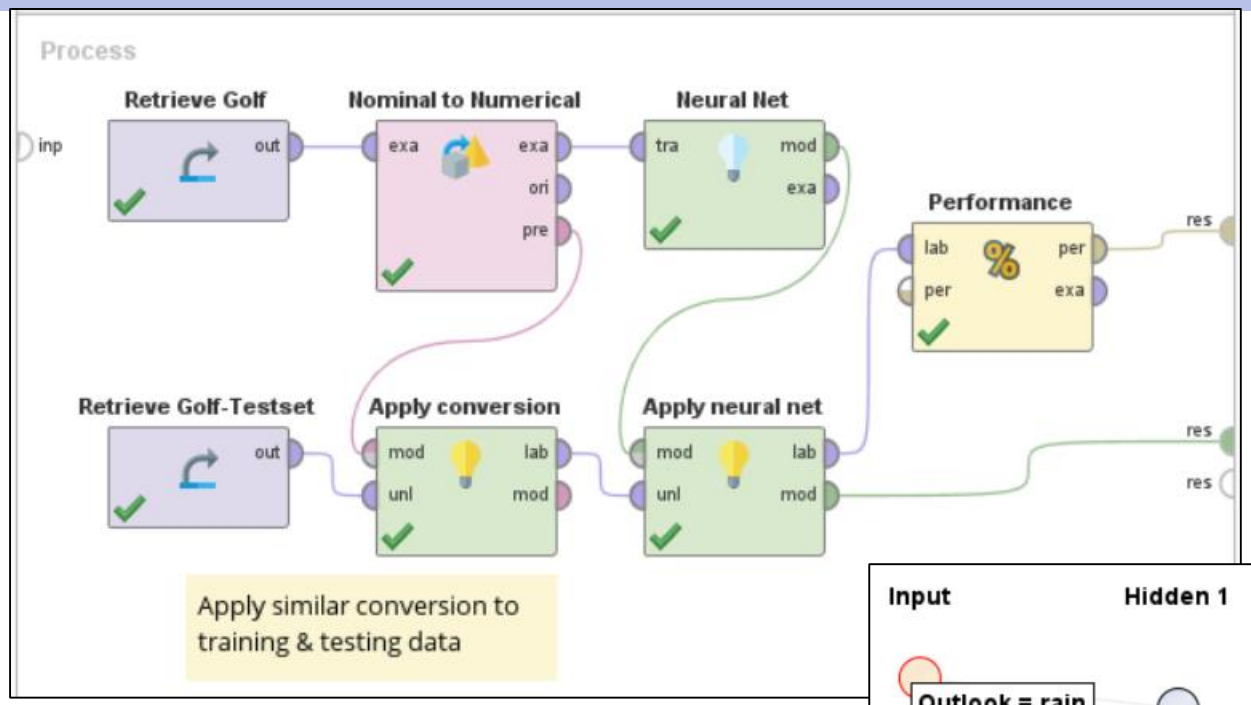
Threshold node is added in every layer

It doesn't take input from nodes in previous layer, but passes thresholds (i.e. weighted biases) to nodes in the next layer

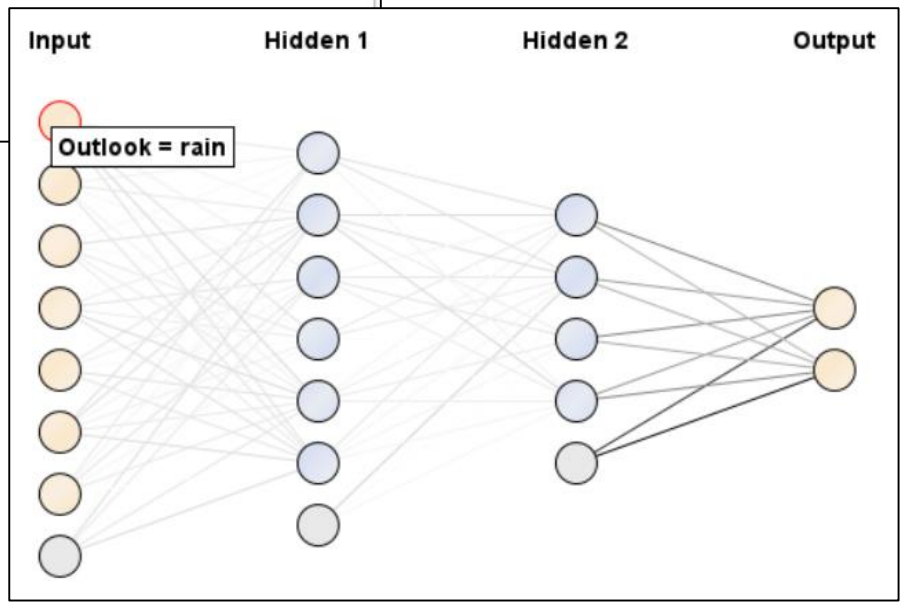
Weights of links from threshold nodes are adjusted during the training, along with other weights

RapidMiner doesn't label output nodes
But we can check them from description

Example (2) : neural net with nominal attributes



hidden layer name	hidden layer sizes
hidden 1	-1 default
hidden 2	4

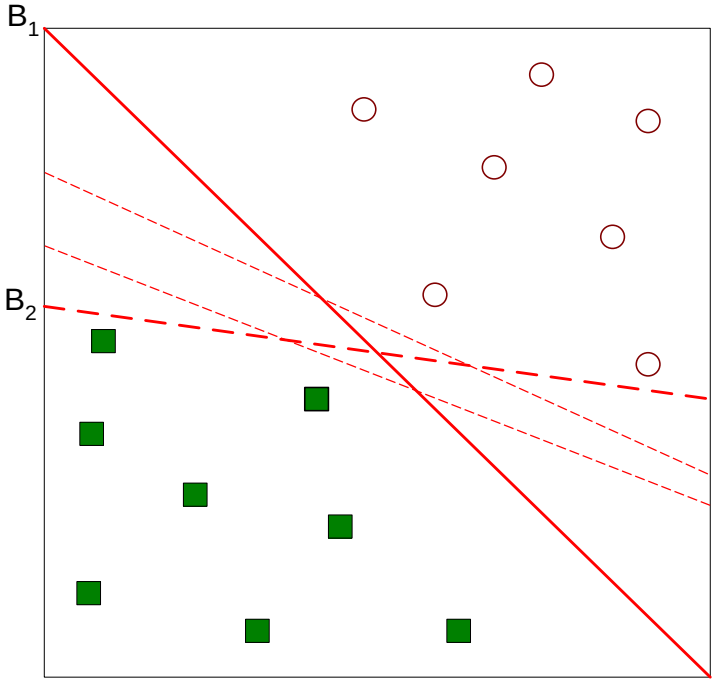


ANN Summary

- Strengths
  - Robust to noises & outliers. Activation functions such as Sigmoid help smoothen data variation (output is bounded)
  - Can handle redundant attributes → links connecting attributes end up with optimal weights that reflect the contribution of each attribute
  - Can handle non-linear & complicated relationship between attributes and class → weights are adjusted to fit patterns in training data
- Weaknesses
  - Training ANN is time consuming
  - Don't guarantee to converge to optimal model. Setting proper parameters is difficult → a lot of trials & errors are needed
  - Support only numeric attributes
  - Unlike trees & rules, ANN model is difficult to explain

Support Vector Machine (SVM)

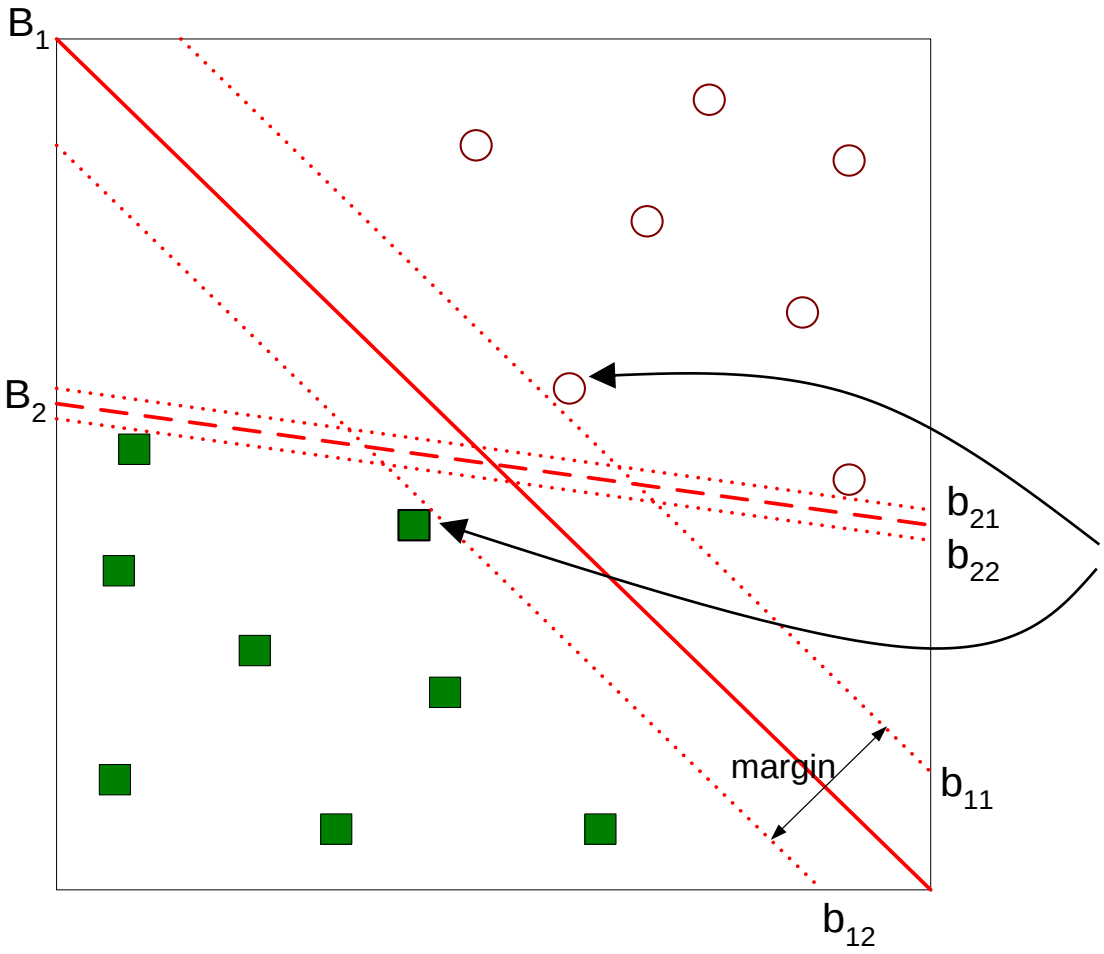
- Find an optimal linear hyperplane (= decision boundary) that separate data into correct classes



- There can be many of them that separates all training records correctly (training error = 0%)
- But may perform poorly on unseen data
- For example, between B_1 and B_2, which one is better ?

Hyperplane's dimension = feature space's dimension - 1
E.g. in 2D feature space, hyperplane is a linear line

Margin of Decision Boundary



Margin of B_1

Find b_{11} and b_{12} → parallel hyperplane on either side of B_1 that touches the closest instances of either class

Support vectors = data points close to hyperplane that influence the position & orientation of the plane (i.e. points on b_{11} and b_{12})

- Decision boundaries with large margins tend perform better on unseen data
  - Small margins are more susceptible to overfitting ➔ any perturbation can have significant impact on the classification
- Objective of SVM training = find decision boundary with maximum margin
  - During the training, parameters (in linear hyperplane function) are adjusted to achieve this objective
  - Because records are not always cleanly separated, some can be on the wrong sides (with penalty added). The optimal hyperplane must also have minimum total penalty

Linearly Separable Data

- Let a data set D consists of tuple (x_i, y_i) where y_i is class label being positive (+1) or negative (-1)
- A decision boundary $\boldsymbol{W \cdot X + b = 0}$ has 2 parameters
  - W = weight vector $(w_1, w_2, \ldots, w_n)$; n = number of attributes
  - b = bias
- Therefore, the prediction y for any data point x

$$y = \begin{cases} 1 & ; W \cdot x + b > 0 \\ -1 & ; W \cdot x + b < 0 \end{cases}$$

- Parallel hyperplanes above & below decision boundary
  - b_{upper} : $W \cdot X + b = 1$
  - b_{lower} : $W \cdot X + b = -1$
  - Margin between them

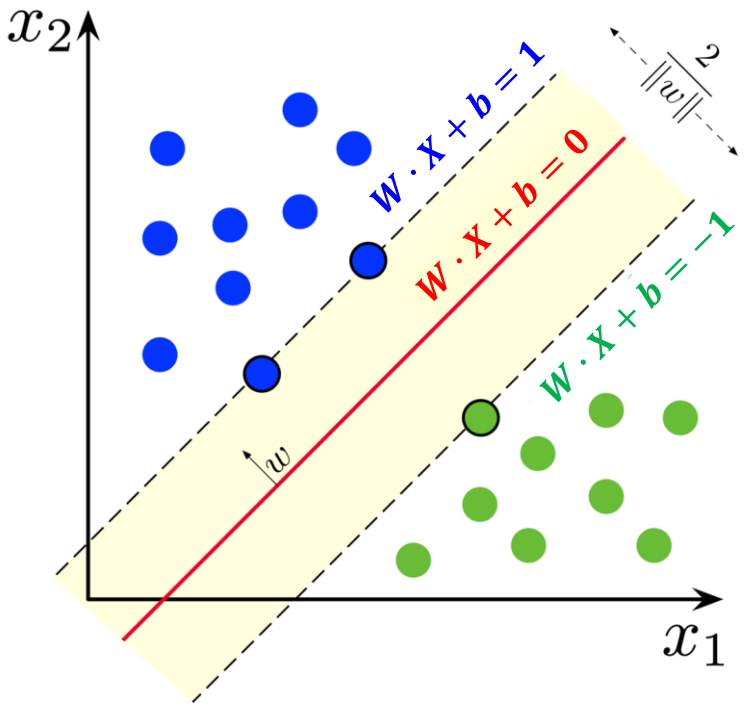
let $p_1 =$ data point on b_{upper}

$p_2 =$ data point on b_{lower}

$$W \cdot (p_1 - p_2) = 2$$

$$\|W\| \times d = 2 \rightarrow \boldsymbol{d = \frac{2}{\|W\|}}$$

$$\|W\| = \text{vector size} = \sqrt{\sum x_i^2}$$



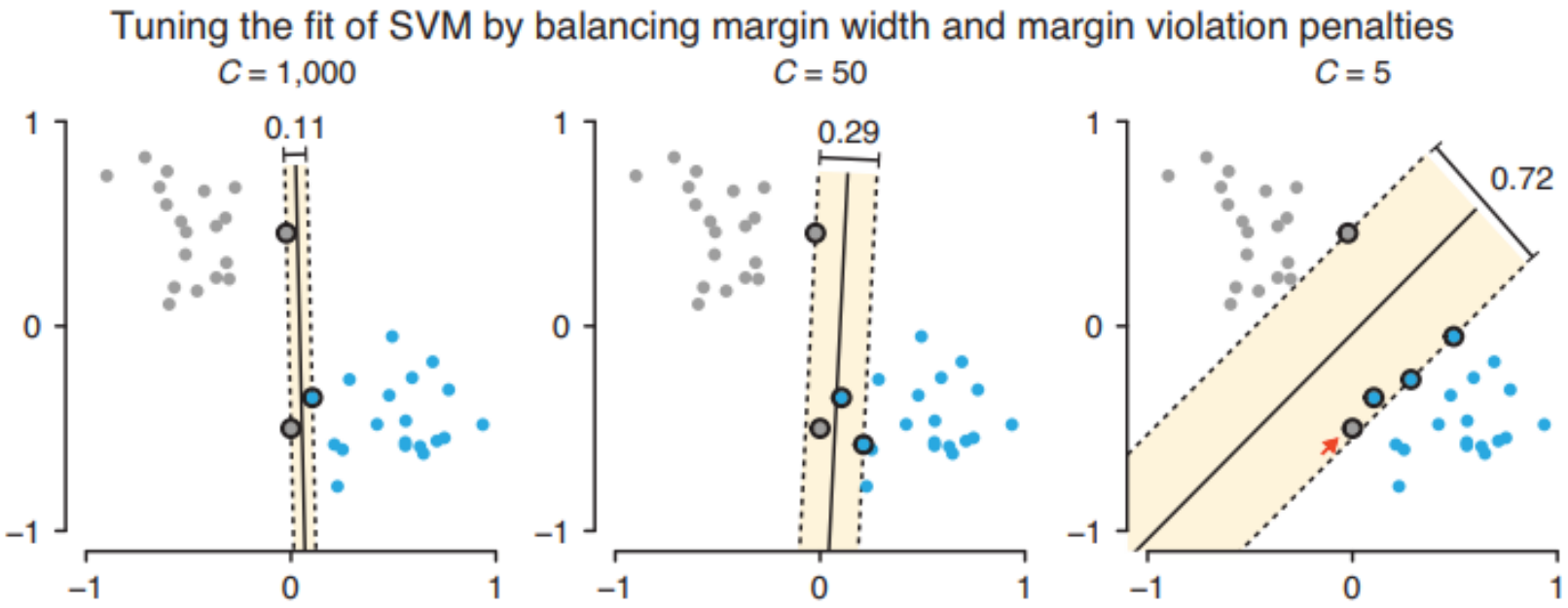
SVM Training

- Given data (x_i, y_i)
  - Find parameters $\mathbf{W}, \mathbf{b}$ that maximize margin $\mathbf{d}$
  - While satisfying the following constraints

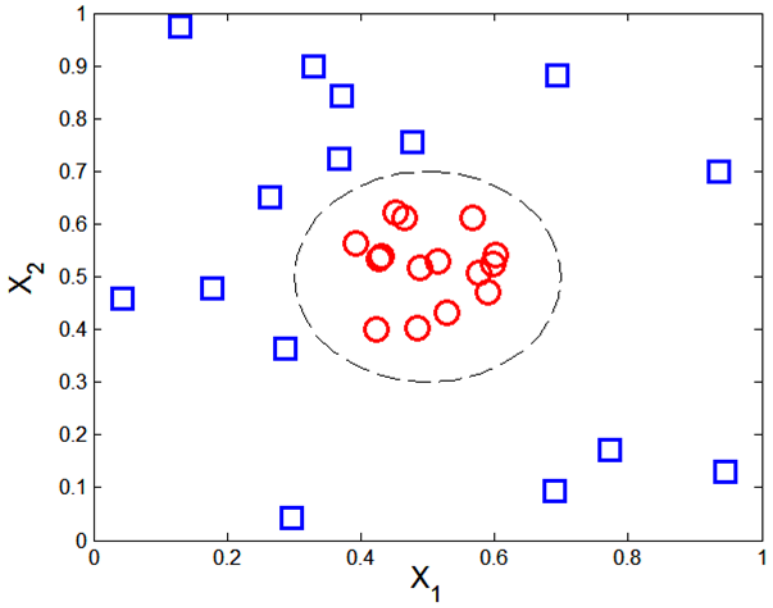
    $W \cdot x_i + b \geq 1$ i.e. above b_{upper} if $y_i = 1$

    $W \cdot x_i + b \leq -1$ i.e. below b_{lower} if $y_i = -1$

    Use a lot of dot-product computation
  - If constraints are not completely satisfied → minimize training error or total penalty. The penalty term is controlled by parameter $\mathbf{C}$
    - Big $\mathbf{C}$ = high penalty for misprediction ➔ narrow margin to avoid misprediction, but may result in overfitting
    - Small $\mathbf{C}$ = low penalty for misprediction ➔ wide margin to allow some misprediction, but may result in underfitting

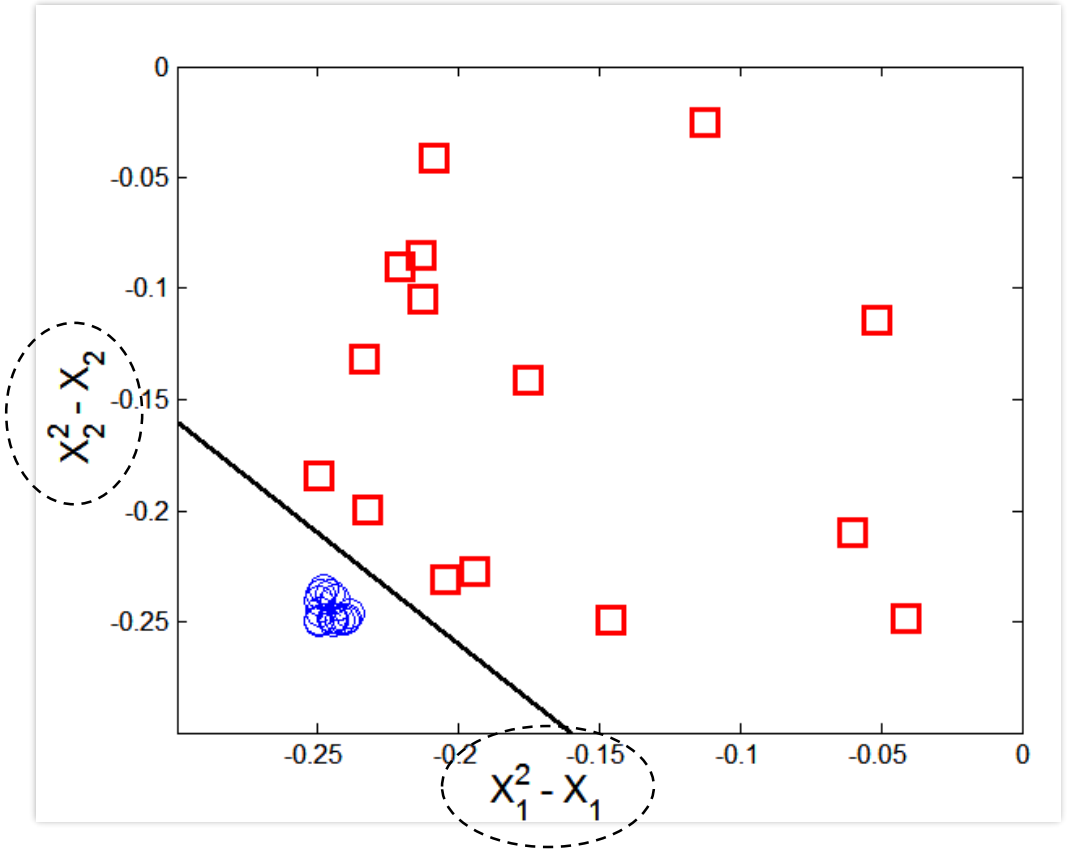

Tuning the fit of SVM by balancing margin width and margin violation penalties
C = 1,000
0.11
C = 50
0.29
C = 5
0.72
1
0
−1
−1
0
1

Nonlinearly Separable Data

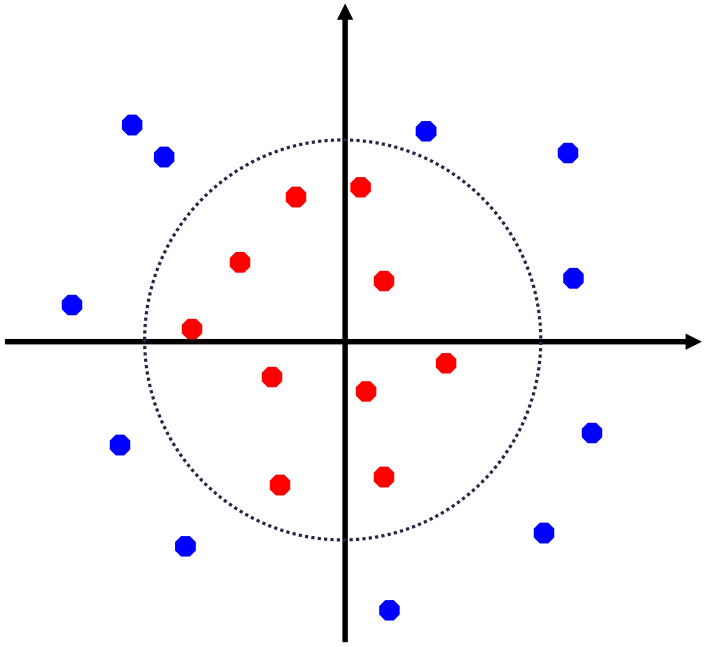


Transformation function

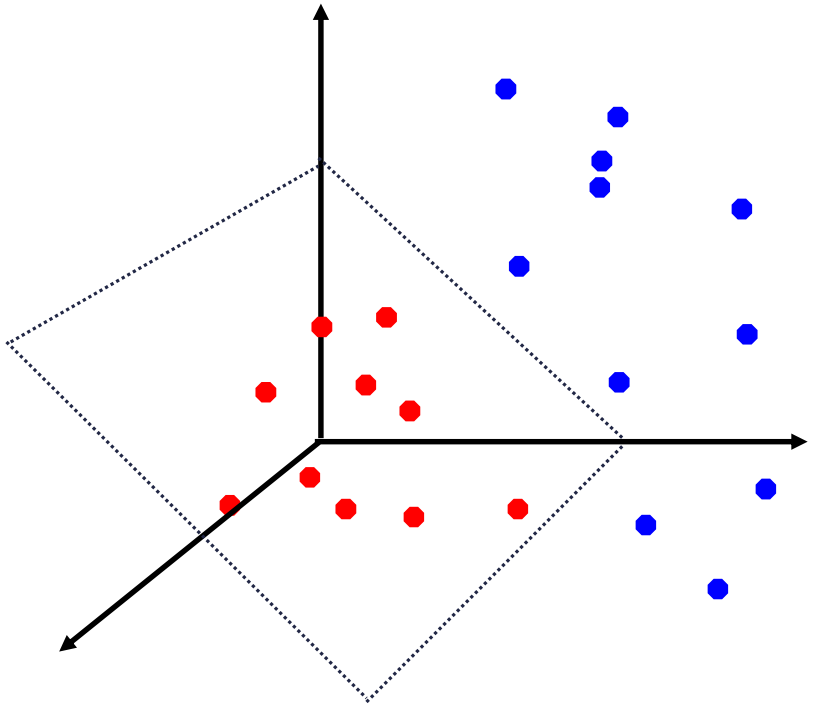
$$\Phi(X) = X^2 - X$$



Transform data into higher dimensional space that is linearly separable



Original feature space X in 2D



New feature space $\Phi(X)$ in 3D

Kernel Functions

- But working with a new space, $W \cdot \Phi(X) + b = 0$, is problematic
  - Difficult to find $\Phi(X)$ that yields linearly separable space
  - Solving optimization may be even more difficult
- Kernel trick
  - SVM computation involves many dot products
  - Consider data points $U = (u_1, u_2, \ldots, u_n)$ and $V = (v_1, v_2, \ldots, v_n)$
    - $U \cdot V$ = cosine similarity = angle between 2 vectors
    - $\Phi(U) \cdot \Phi(V)$ = the same angle between 2 vectors
  - If we have a kernel function $K(U, V) = \Phi(U) \cdot \Phi(V)$ ➔ dot product in a new space can be computed using values in the original space & we don't need to care about $\Phi(X)$
  - Solving optimization by using original values is easier

- Instead of calculating $W \cdot \Phi(X) + b = 0$
- We can use kernel $K(W, X) + b = 0$

Applying Φ to W or not doesn't matter because our goal is to find optimal weights for weight vector W

- In summary, SVM training can be thought of as
  - Transforming data from original space (nonlinear) to new space (linear)
  - Finding optimal hyperplane
  - Transforming data back to original space
  - But the transformation doesn't actually occur. We achieve this effect by using kernel trick

- Some kernels that satisfy $K(U,V) = \Phi(U) \cdot \Phi(V)$
  - Dot product $K(U,V) = U \cdot V$
  - Polynomial degree d $K(U,V) = (U \cdot V + 1)^d$
  - Radial basis function (RBF) $K(U,V) = e^{-\gamma\|U-V\|^2}$
  - Sigmoid or tanh $K(U,V) = \tanh(k\, U \cdot V - \delta)$
- Most kernels are complicated & require a lot of parameters
  - Dot product or polynomial (degree 1 or 2) is recommended for simple classification

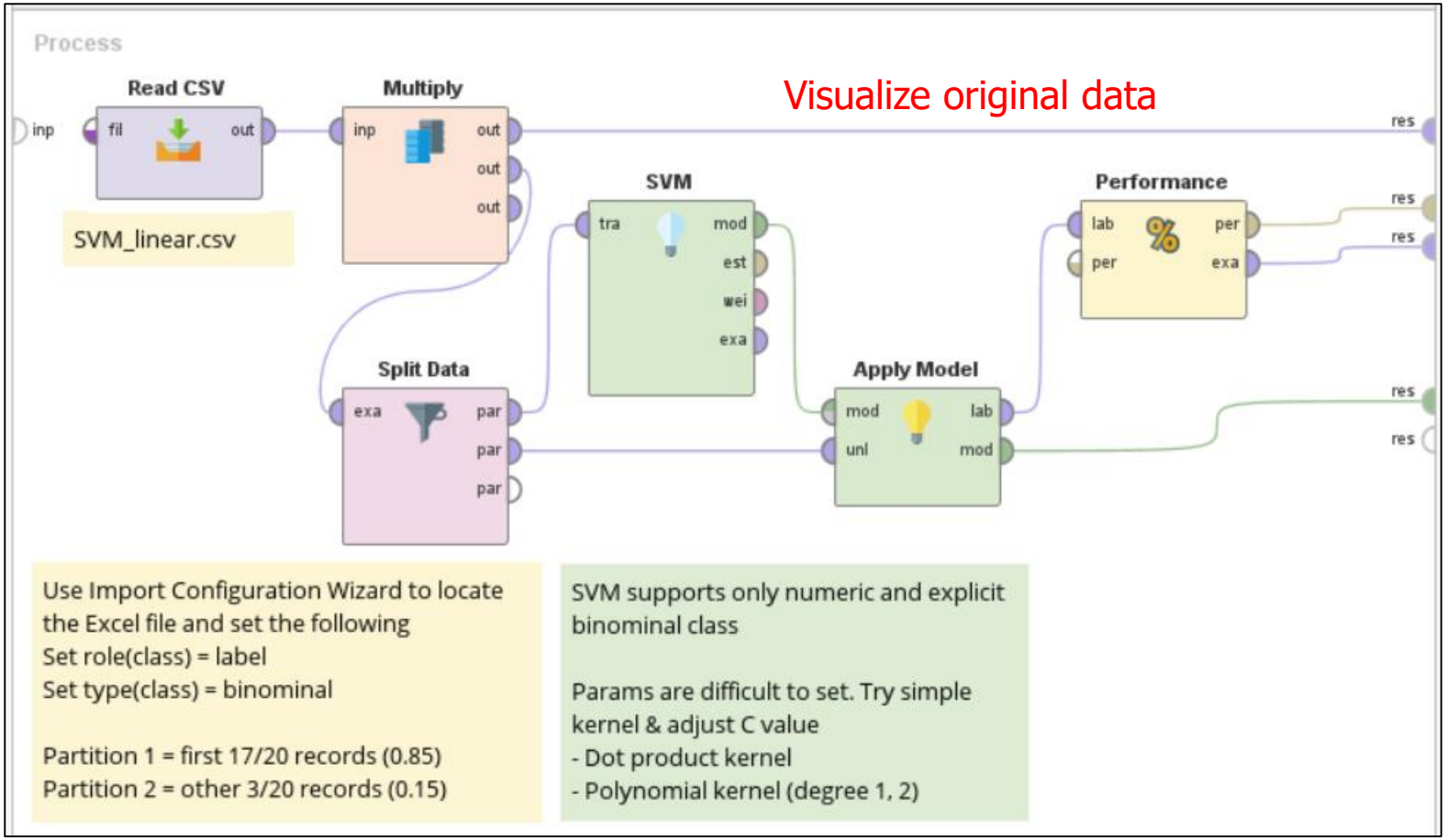
Famous Implementations

mySVM	Basic implementation Support only binary classification
LibSVM	More powerful implementation ○ C-SVC and nu-SVC modules for classification, using internal 1-against-1 training for multi-class problem ○ Epsilon-SVR and nu-SVR modules for regression

- C-SVC uses parameter C (any positive value) to control penalty function
- Nu-SVC (ν-SVC) uses parameter ν (between 0-1) to control #support vectors and #mispredictions. Suppose that $\nu = 0.1$
  - At least 10% of data are support vectors, i.e. touching boundaries
  - At most 10% of data are on the wrong side of hyperplane

Too many support vectors may lead to overfitting model

Example (3) : SVM with linear data



Kernel Model

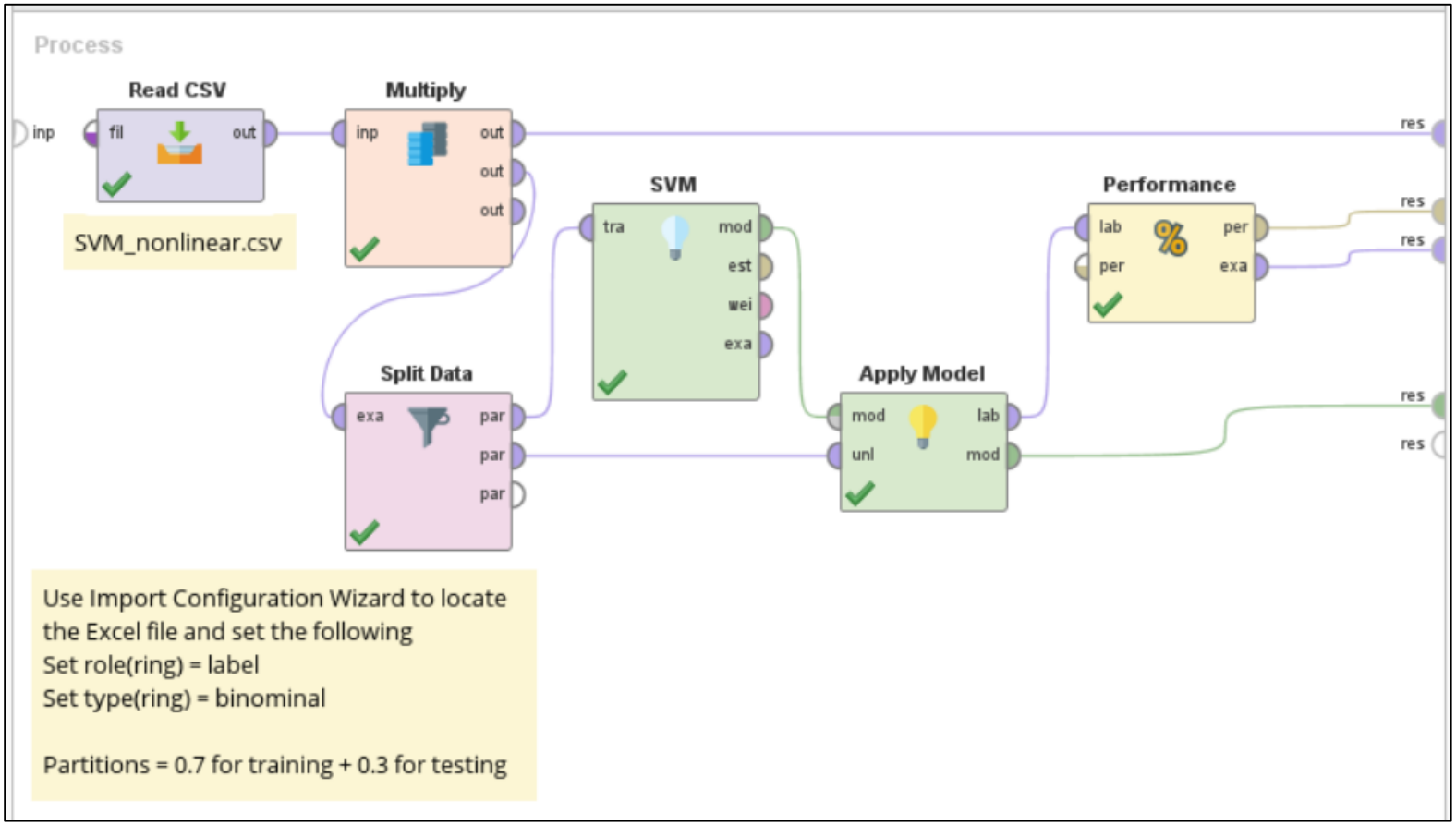
```
Total number of Support Vectors: 17
Bias (offset): 0.051

w[x1] = 0.370
w[x2] = 1.351
```

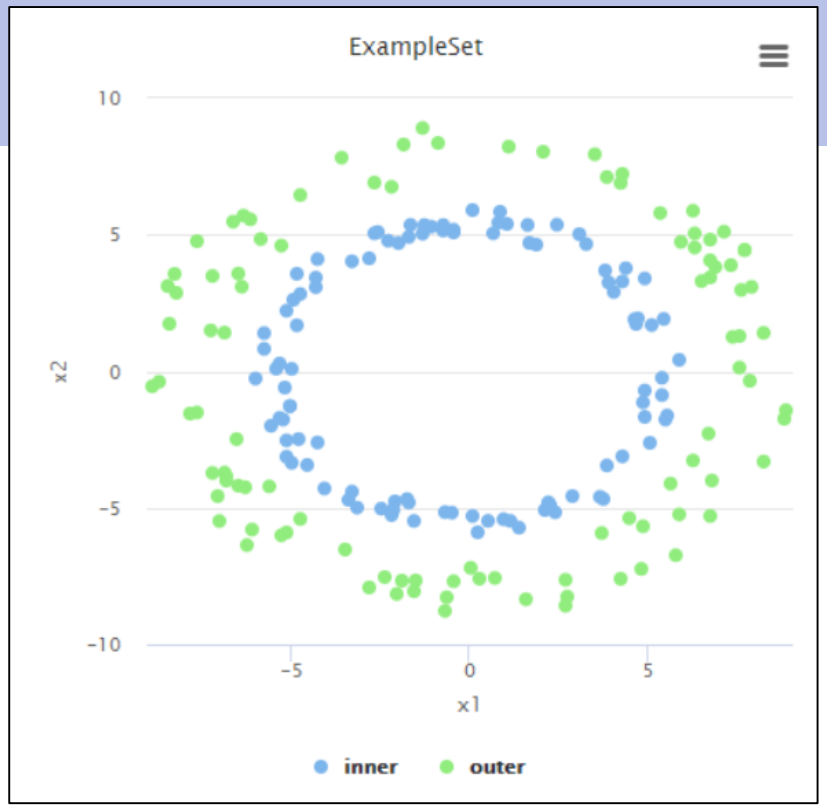
Weight of each attribute can be used for attribute scoring & selection (see Chapter 3)

Row No.	class	prediction(class)	confidence(A)	confidence(B)	x1	x2
1	A	A	0.879	0.121	1.500	1
2	B	B	0.466	0.534	1.500	4
3	B	B	0.074	0.926	2	7

Testing #2 is close to hyperplane and thus difficult to predict. Confidence of both classes are very close

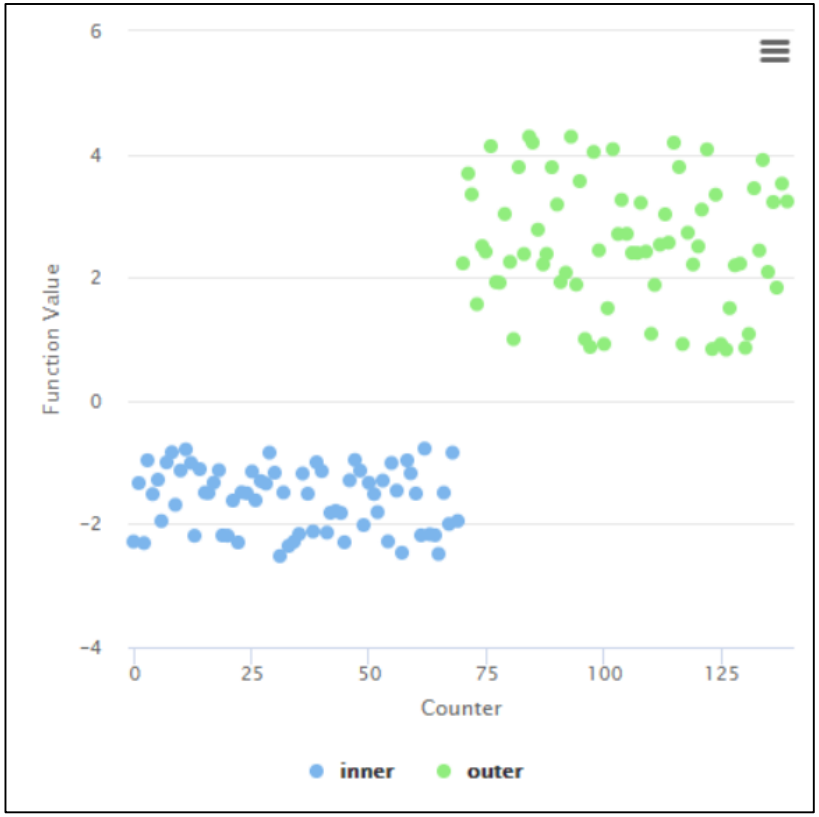
Example (4) : SVM with nonlinear data



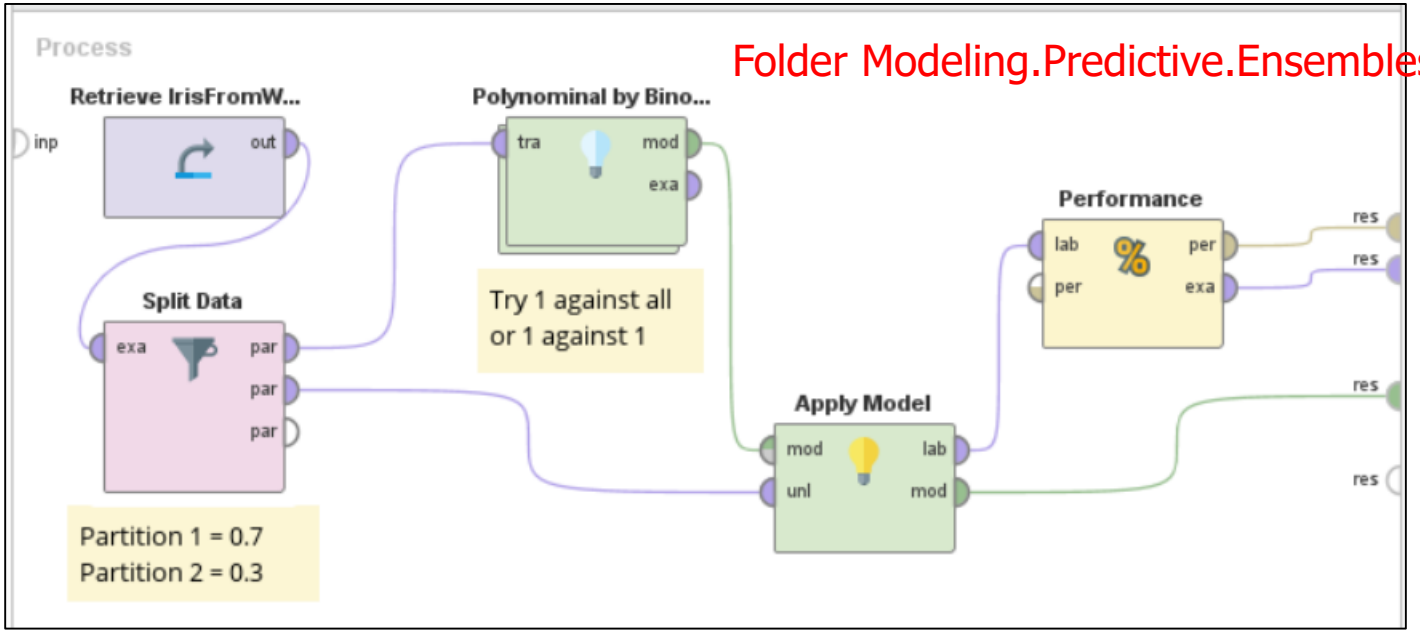


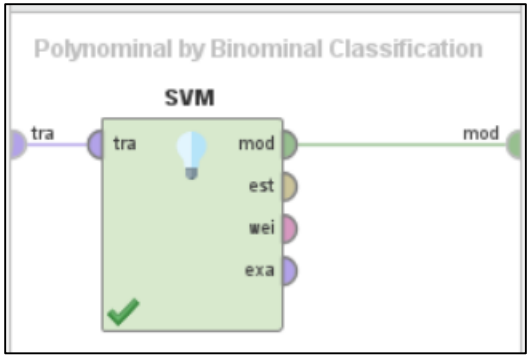
Original data (non-linear)

Support vector visualization (dot Kernel)
Try using other kernels

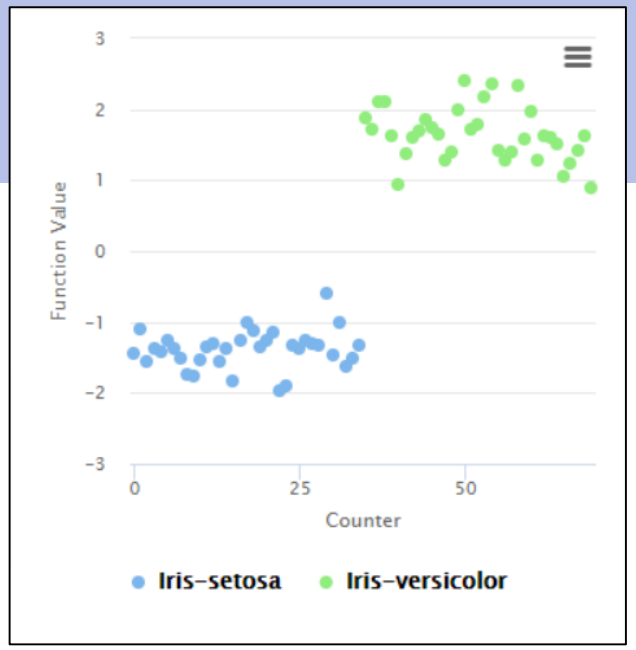


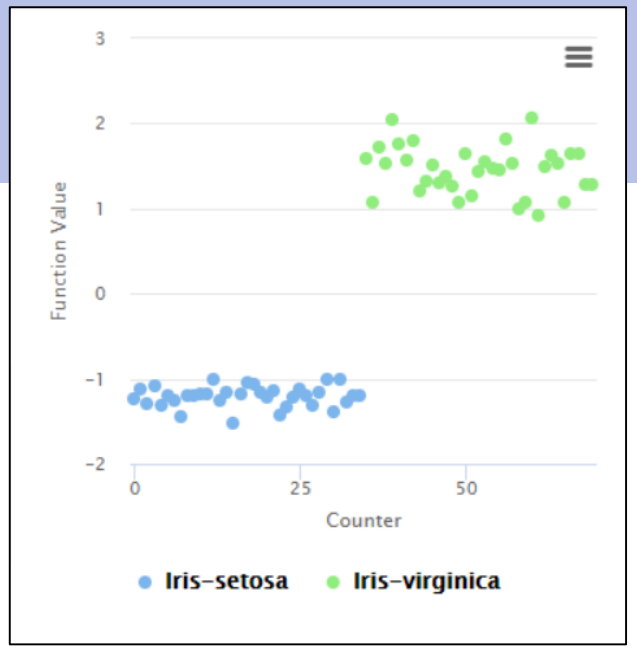
Example (5) : SVM with polynominal class

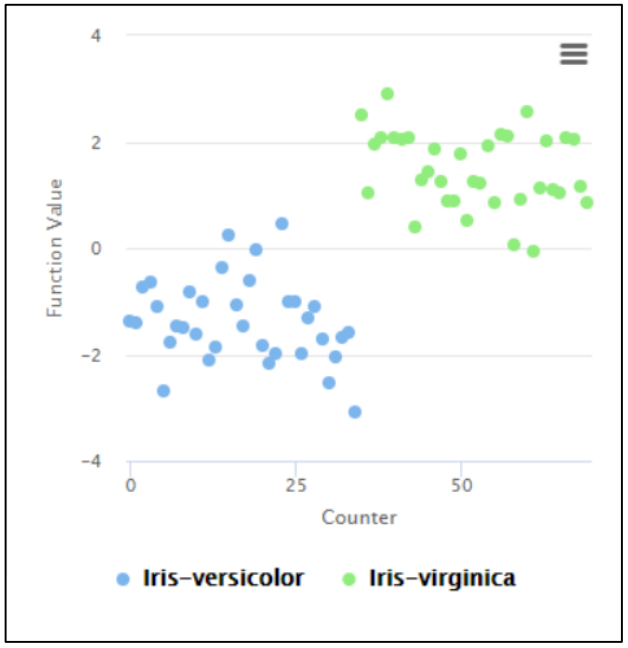




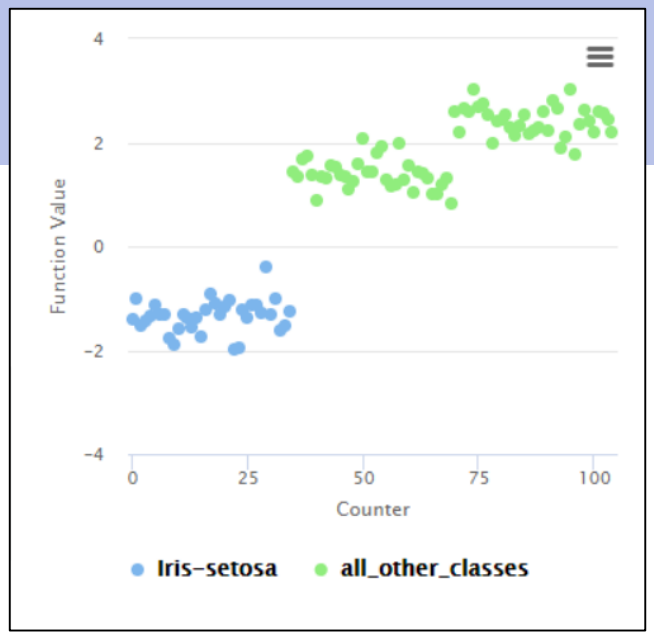
Train multiple binary classifiers and use all of them to predict a new record (see Chapter 9 for ensemble classification)

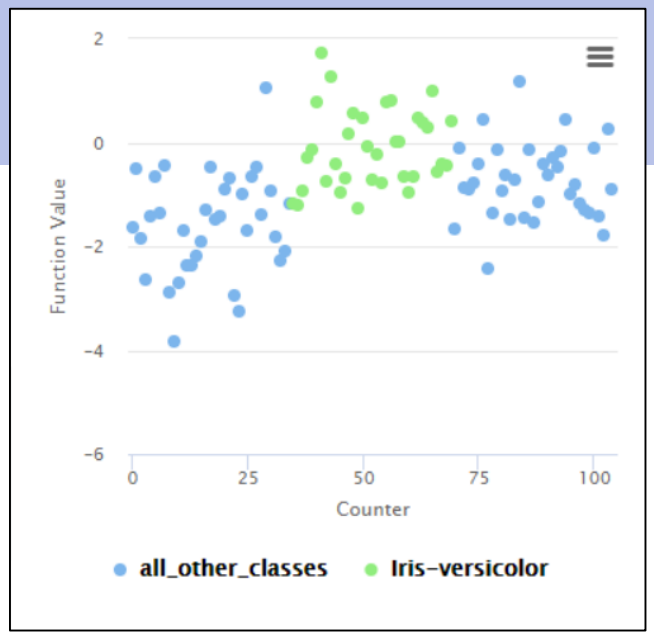


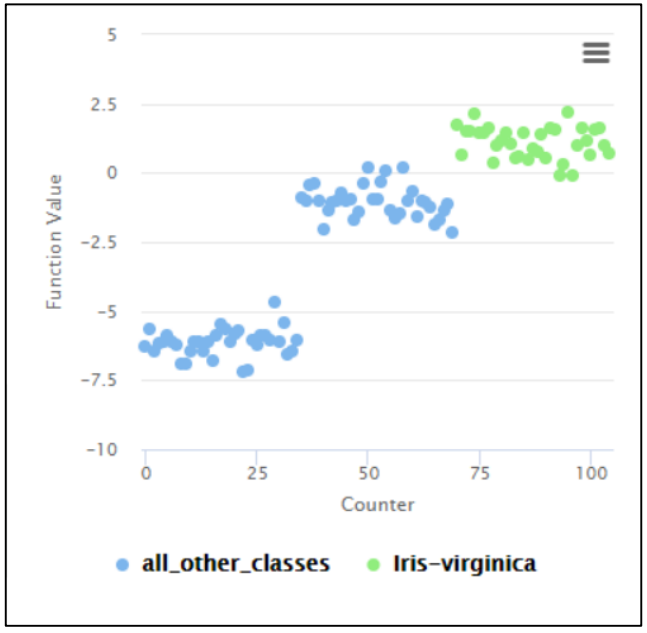




1-against-1 binary classifiers

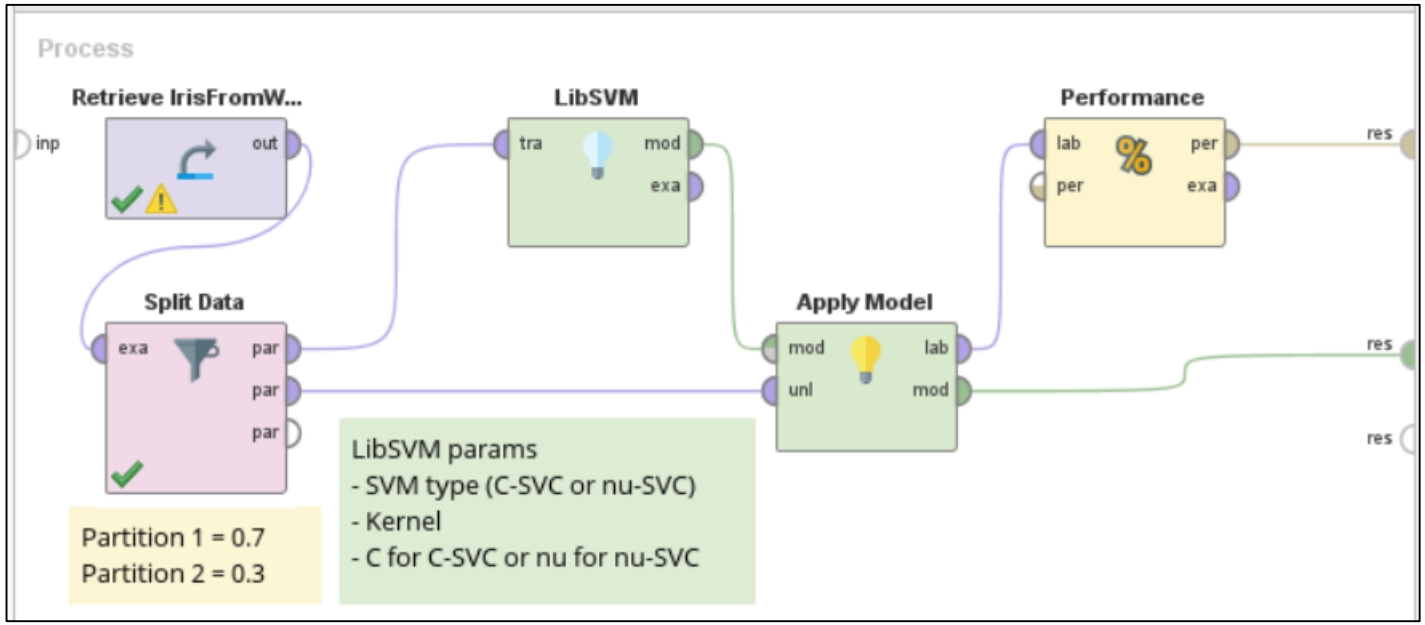






1-against-all binary classifiers

Example (6) : LibSVM



SVM Summary

- Strengths
  - Work well with high dimensionality data (i.e. many attributes)
  - Can handle non-linear decision boundary & complicated relationship between attributes and class → kernel + optimal hyperplane
  - Less prone to overfitting
  - Better than neural network at finding global optimum
  - Flexible & performance is often better than many other classifiers
- Weaknesses
  - High computation overhead
  - Difficult parameter setup
  - All attributes must be numeric
  - Support only binary classification